



计算机操作系统

倪福川

fcni_cn@mail.hzau.edu.cn

华中农业大学 信息学院

2025/11/10





1、UNIX的进程状态及其转换

• 9种状态:

- | | |
|-------------|---------|
| • 用户模式下运行 ; | 内核模式下运行 |
| • 在存储器中就绪; | 在存储器中睡眠 |
| • 就绪, 被交换; | 睡眠, 被交换 |
| • 被剥夺; | 创建 |
| • 僵死 | |

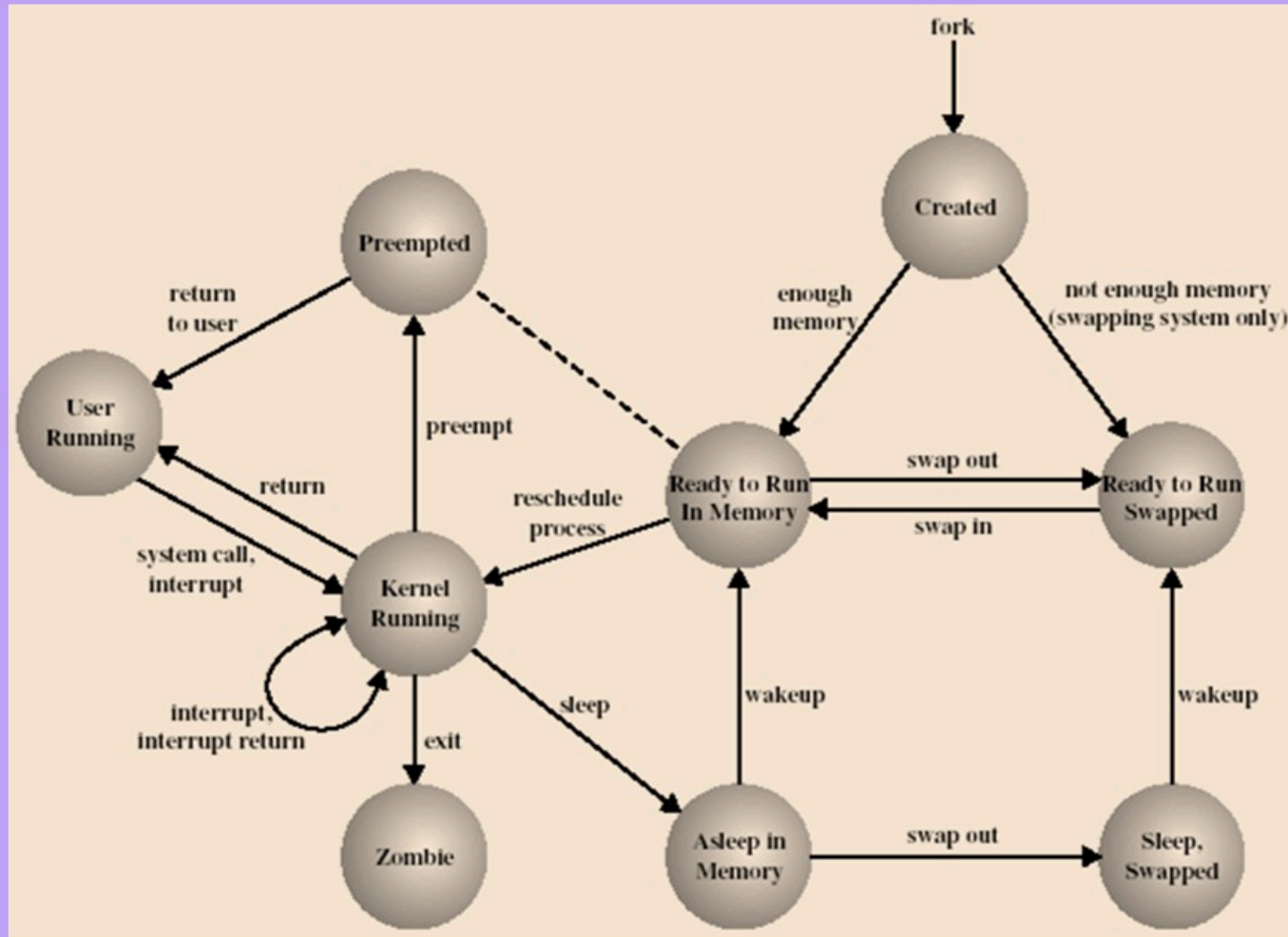
Unix采用两个状态表示进程在用户模式下还是在内核模式下执行:

用户态/ 内核态

状态“被抢先（被剥夺）”与“内存就绪”的地位相同，要等到下一次进程调度时，才能回到“用户态执行”。

正在核心态运行的进程不能被剥夺，仅当它即将返回用户态时，才能被剥夺







2、进程操作—创建

- 进程在执行过程中，能通过系统调用创建多个进程
- 子进程资源的获得：
 - 物理资源（CPU时间、内存、I/O设备、文件）
 - 从OS那里获得
 - 从其父进程资源子集中获得（可以防止超载）
 - 初始化数据（或者输入数据）
- 创建新进程后：
 - 父进程与子进程并发执行
 - 父进程等待，直到某个或者全部子进程执行完毕。





2、进程操作—UNIX的进程控制

UNIX的进程控制的系统调用：

`fork()` : 创建新进程

`exec()::` 执行可执行程序

`exit()` : 终止

`sleep()` : 暂停一段时间

`pause()`: 暂停并等待信号

`wait()` : 等待子进程暂停或终止

`kill()` : 发送信号到某个或一组进程

`ptrace()` : 设置执行断点(breakpoint),
允许父进程控制子进程的运行





fork()—创建一个新进程

- fork()调用格式: `pid = fork()`

在调用fork () 之后, 父进程和子进程均在下一条语句上继续运行。

可以检查fork () 返回值, 同时进程可以决定下面要执行的代码。

□ 父、子进程的**fork()**返回值不同

在子进程中返回时, pid为0;

在父进程中返回时, pid为所创建的子进程的标识。





fork()—创建新进程

- 除了子进程标识符和其**proc**结构中的某些特性参数不同之外，子进程是父进程的**精确复制**。
- 创建子进程后，执行**返回父进程**，或**调度切换**到子进程或其他进程。
- 父、子进程的运行是无关的，运行顺序也不固定。





父进程

```
main()
{
    pid_t val;
    printf("PID...");
    val=fork();
    if(val!=0)
        printf("parent...");
    else
        printf("child...");
}
```

执行

分裂

main()

父进程

```
{
    pid_t val;
    printf("PID...");
    val=fork();
    if(val!=0)
        printf("parent...");
    else
        printf("child...");
}
```

val = 0

继续
执行

main()

子进程

```
{
    pid_t val;
    printf("PID...");
    val=fork();
    if(val!=0)
        printf("parent...");
    else
        printf("child...");
}
```

val > 0

继续
执行



主观题 10分



fork()调用输出 例子（1）

```
#include<stdio.h>
```

```
#Include<sys/types.h>
```

```
#include<unistd.h>
```

/*fork系统调用的一个简单例子（不含出错检查）*

```
void main(void) {  
    printf("Hello \n");  
    fork();  
    printf("Bye \n") ;  
}
```

运行结果

Hello
Bye
Bye



单选题 10分



fork()调用例子 (2)

```
void main(void) {  
    int i = 0;  
    for(; i<2; i++){  
        pid_t pid = fork();  
        if (pid < 0 ){  
            Printf("fork error\n"); Exit(0); }  
        if(pid == 0){  
            printf("LOOP%d, pid %d,Parent Proc  
%d",i,getpid(),getppid()); }  
        else{  
            printf("LOOP%d, pid %d,Parent  
Proc %d",i,getpid(),getppid()); }  
        }  
    }  
    return 0; }
```

程序执行后 有()次输出?

- ☐ A 2 ☐ B 4 ☐ C 6 ☐ D 3



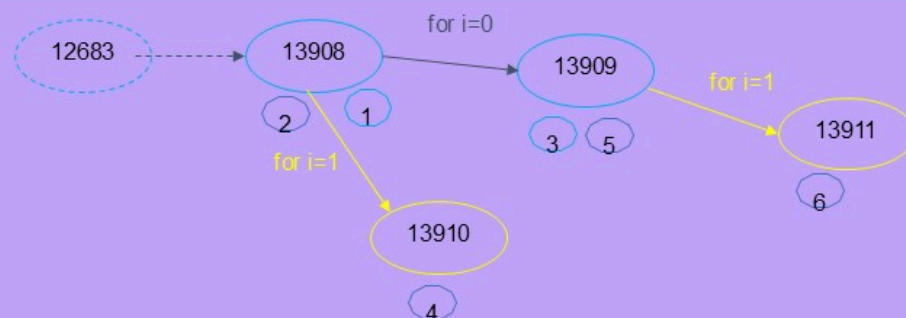


fork()调用-for 循环 解析

```
admin@HUAWEI2288V5:~/os-code$ gcc -o fork-test fork-test.c
admin@HUAWEI2288V5:~/os-code$ ./fork-test
Loop0:pid 13908,Parent proc:12683
Loop1:pid 13908,Parent proc:12683
Loop0:pid 13909,Parent proc:13908
Loop1:pid 13910,Parent proc:13908
Loop1:pid 13909,Parent proc:13908
Loop1:pid 13911,Parent proc:13909
```

PStree

```
—bash—fork-test—fork-test—fork-test
                  |
                  —fork-test
```





fork()调用例子（2）

```
#include<stdio.h>
#include<sys/types.h>
#include<unistd.h>
void main(void){
    if (fork()==0)
        printf("In the CHILD process\n");
    else
        printf("In the PARENT process\n");
}
```

如果多次运行该程序，有时语句
“In the CHILD process” 会在
In the PARENT process”之前，
有时却相反。

程序的运行无法保证输出顺序，输出顺序依赖于内核所用的调度算法





fork()调用例子（3）

```
#include<stdio.h>
#include<sys/types.h>
#include<unistd.h>
void main(void) {
    int i;
    Static char buffer[10];
    if (fork()==0){          /* 在子进程中 */
        strcpy(buffer,"Child\n");}
    else{                    /*在父进程中*/
        strcpy(buffer,"Parent\n");
    }
    for(i=0;i<5;++i){       /*两个进程均执行此语句*/
        Sleep(1);           /*每次5遍*/
        write(1,buffer,sizeof(buffer));
    }
}
```

运行结果

Parent
Child
Parent
Child
Parent
Child
Parent
Child





fork()调用例子 (4)

```
#include<stdio.h>
#include<sys/types.h>
#include<unistd.h>
int global=4;
void main(void)
{
    int pid;
    int vari=5;
    printf("before fork\n");
    if ((pid=fork())<0){
        Printf("fork error\n");
        Exit(0);}
    else if(pid==0){
        global++;
        vari--;
        printf("Child changed the vari
        and global.\n");
    }
    else
        Printf("Parent didn't changed
        the vari and global.\n");

    printf("global=%d,vari=%d\n",gl
    obal,vari);
    Exit(0);
}
```





运行结果1

global=5,vari=4;

global=4,vari=5;

运行结果2

global=4,vari=5;

global=5,vari=4

分析：子进程和父进程的数据段、堆、栈段不同。

子进程执行语句“**global++**，**vari--**”时，只是对自己的数据段进行了修改，并未影响到父进程的数据段，





2、进程操作—终止

- 进程终止：Unix中系统调用`exit()`
- 进程终止时：返回数据到其父进程，它相关所有资源：内存、打开文件和I/O缓冲会被释放掉。
- 父进程可通过`wait ()`等待子进程的结束，`wait ()`可以将子进程的标识符返回给父进程。





C程序内存布局

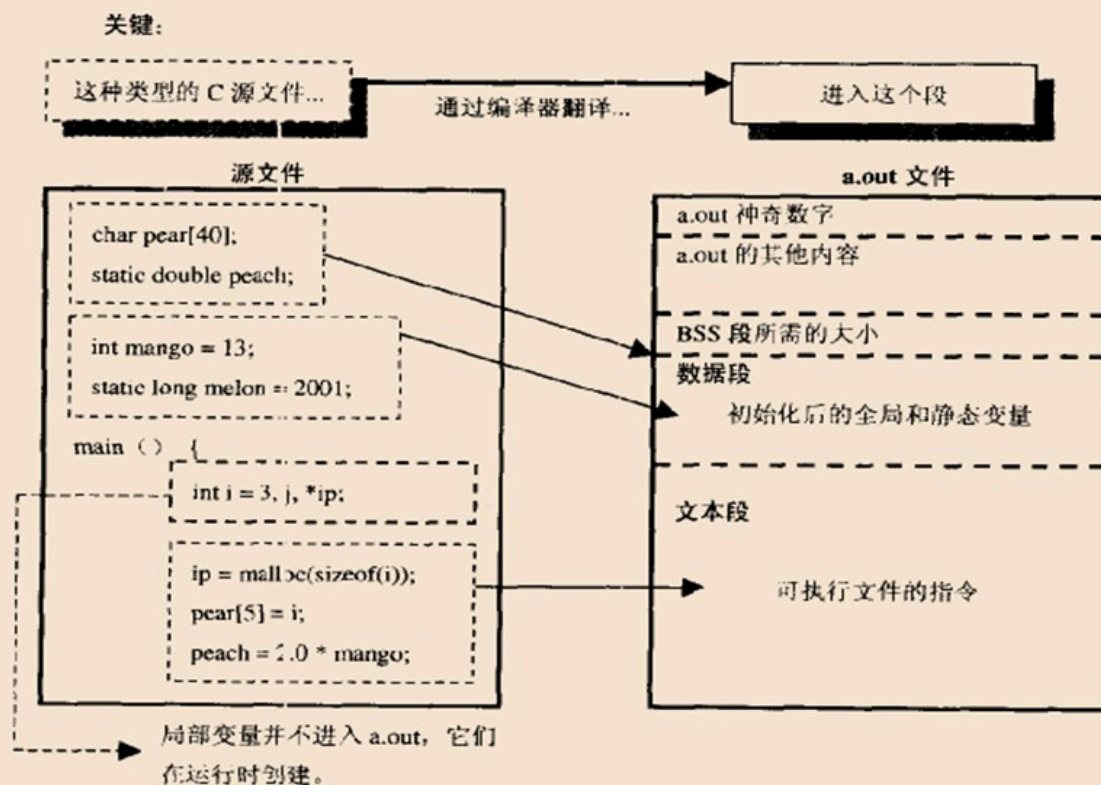
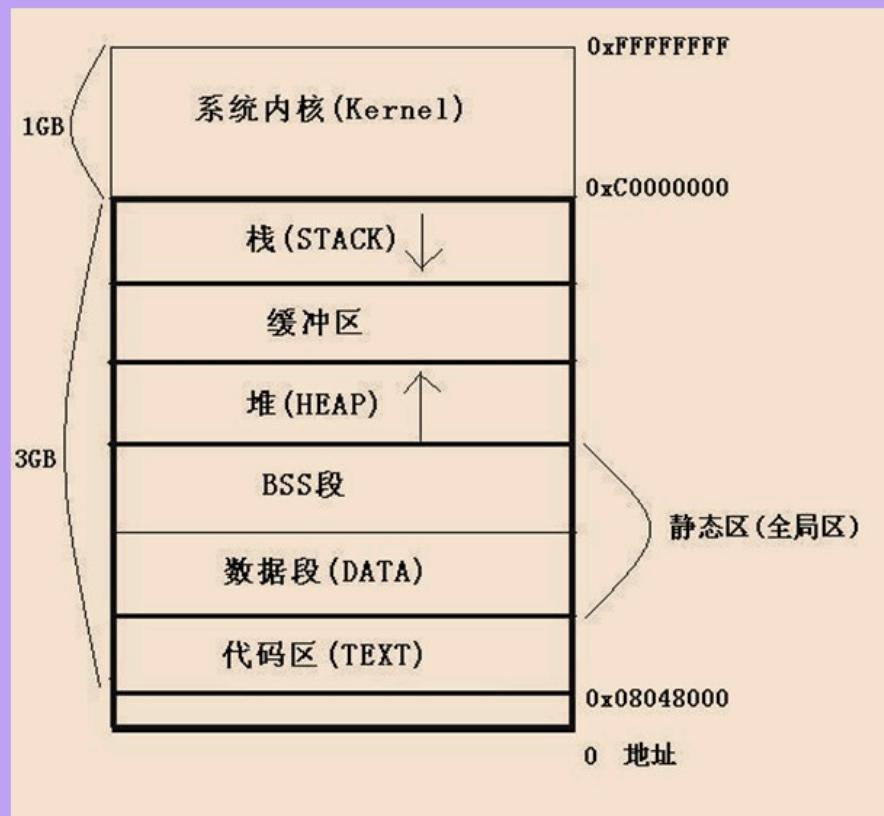
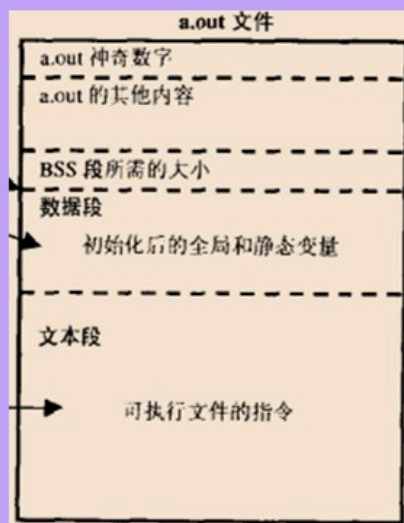


图 6-1 C 语句的各部分会出现在哪些段中





C程序内存布局





3、进程间通信机制—IPC

- 操作系统提供使协作进程通过进程间通信（IPC）工具来进行彼此间的通信。
 - 管道
 - 消息
 - 共享存储区
 - 信号
 - 信号量





进程间通信机制— UNIX消息队列API

system v接口**API**

- `msgget` 依据用户给出的整数值`key`，创建新消息队列或打开现有消息队列，返回一个消息队列ID；
- `msgsnd` 发送消息；
- `msgrcv` 接收消息，可以指定消息类型；没有消息时，返回-1；
- `msgctl` 对消息队列进行控制，如删除消息队列；





```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>

#define MAX_TEXT 512
struct msg_st //消息队列的结构体
{
    int my_msg_type;
    char msg_text[BUFSIZ];
};
```

```
int main(int argc, char **argv)
{
    pid_t pid;
    int i = 1;
    int status;
    printf("pid %d, fork \n", getpid());
    if ((pid = fork()) == -1)
    {
        perror("fork");
        exit(EXIT_FAILURE);
    }
    else if (pid == 0) //child procc
    {
        send();
    }
    else //parent proc
    {
        receive();
    }

    wait(); //
    return 0;
}
```

```
int send()
{
    struct msg_st some_data;
    int msgid;
    char buffer[BUFSIZ];
    if ((msgid = msgget((key_t)12345, 0666 | IPC_CREAT)) == -1)
    {
        perror("msgget");
        exit(EXIT_FAILURE);
    }
    printf("PID %d, Enter the mssage to send:", getpid());
    fgets(buffer, BUFSIZ, stdin);
    some_data.my_msg_type = 1;
    strcpy(some_data.msg_text, buffer);
    if ((msgsnd(msgid, (void *)&some_data, MAX_TEXT, 0)) == -1)
    {
        perror("msgsnd");
        exit(EXIT_FAILURE);
    }
    return 0;
}
```

```
int receive()
{
    int msgidl;
    struct msg_st some_data1;
    int msg_to_recevie = 0;
    if ((msgidl = msgget((key_t)12345, 0666 | IPC_CREAT)) == -1)
    {
        perror("msgget");
        exit(EXIT_FAILURE);
    }
    if (msgrcv(msgidl, (void *)&some_data1, BUFSIZ, msg_to_recevie, 0) == -1)
    {
        perror("msgrcv");
        exit(EXIT_FAILURE);
    }
    printf("PID %d, recevier mssage : %s", getpid(), some_data1.msg_text);
    if (msgctl(msgidl, IPC_RMID, 0) == -1)
    {
        fprintf(stderr, "msgctl(IPC_RMID) failed \n");
        exit(EXIT_FAILURE);
    }
    return 0;
}
```



Linux操作系统编程



中国大学MOOC

课程 ▾

学校

学校云

慕课堂

下载APP

搜索感兴趣的

首页 > 计算机



Linux操作系统编程

第4次开课 ▾

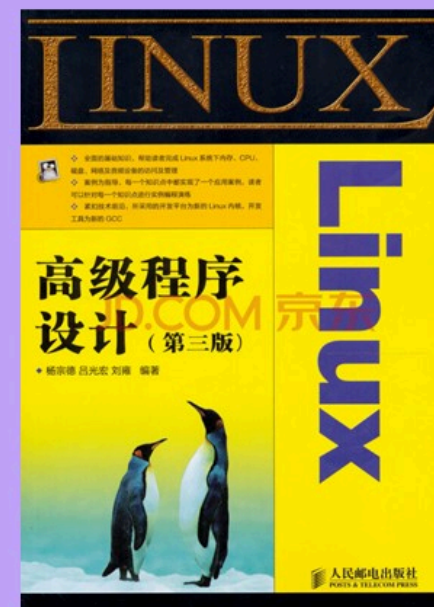
开课时间: 2020年03月09日 ~ 2020年07月14日

学时安排: 2

当前开课已结束

立即自学

自学模式不提供结课证书, 想参加下学期课程, 点击



主观题 10分



在本课堂上，你想学习那些内容？
你有那些建议？



投票(匿名) 最多可选5项



在线听课评价:

- A 教师准备充分,讲述条理清晰
- B 管理课堂不够, 迟到、缺课人数多
- C 教学方式单一、缺乏互动
- D 学生的收获少, 获得感不强
- E 本课堂 有些内容没听懂, 还需消化

